NBA Knowledge Graph Pipeline

Web Data Mining and Semantics – Final Project

DIA – A4

From Web Crawling to RAG-based Question Answering

Yijia ZENG

Thomas JIN

March 25, 2026

Contents

1 Data Acquisition and Information Extraction **2**
1.1 Domain and Seed URLs . . . 2
1.2 Crawler Design and Ethics . . . 2
1.3 Cleaning Pipeline . . . 2
1.4 Named Entity Recognition . . . 2
1.5 Ambiguity Cases . . . 3

2 Knowledge Base Construction and Alignment **3**
2.1 RDF Modeling Choices . . . 3
2.2 Entity Linking and Alignment . . . 3
2.3 SPARQL Expansion Strategy . . . 4
2.4 Final KB Statistics . . . 4

3 Reasoning with SWRL **5**
3.1 SWRL Rules on family.owl . . . 5
3.2 SWRL Rules on the NBA Knowledge Graph . . . 6

4 Knowledge Graph Embeddings **6**
4.1 Data Preparation and Splits . . . 6
4.2 Models and Training . . . 7
4.3 Evaluation Metrics . . . 7
4.4 t-SNE Visualizations and Nearest Neighbours . . . 7

5 RAG over RDF/SPARQL **9**
5.1 Schema Summary and Prompt Design . . . 9
5.2 Self-Repair Mechanism . . . 9
5.3 Baseline vs RAG Evaluation . . . 10

6 Critical Reflection **11**
6.1 KB Quality and Noise . . . 11
6.2 Alignment Limitations . . . 11
6.3 Rule-based vs Embedding-based Reasoning . . . 11
6.4 RAG Limitations and Improvements . . . 11

1 Data Acquisition and Information Extraction

1.1 Domain and Seed URLs

We chose NBA basketball as the domain. It has a lot of structured information available online, including players, teams, awards, and seasons, which makes it a good case for knowledge graph construction. Entities have clear types and are well connected across many sources.

The crawler was seeded with the following representative URLs:

- `https://en.wikipedia.org/wiki/LeBron_James`
- `https://en.wikipedia.org/wiki/NBA_Most_Valuable_Player_Award`
- `https://www.basketball-reference.com/players/`
- `https://www.nba.com/news/`
- `https://www.espn.com/nba/`

From these seeds, the crawler follows internal links to collect additional NBA-related pages.

1.2 Crawler Design and Ethics

The crawler is built with `requests` for HTTP requests and `trafilatura` for clean text extraction from HTML. Starting from the seed URLs, it follows links up to a defined depth and stores raw text output as `data/raw/crawler_output.jsonl`.

We applied the following ethical constraints:

- A delay of at least 1 second between requests to avoid overloading servers.
- Only public pages are crawled; no login, paywall, or authentication bypass.
- We manually checked that the seed domains do not prohibit automated access in their `robots.txt`.
- The crawl scope is limited to NBA-related content; we do not collect personal data.
- All collected data is used exclusively for academic purposes.

1.3 Cleaning Pipeline

After crawling, the cleaning step (`src/crawl/clean.py`) applies the following steps:

1. Remove duplicate pages (by URL and by content hash).
2. Filter out short texts with fewer than 100 characters (likely navigation menus or errors).
3. Strip any remaining HTML tags not handled by trafilatura.

The cleaned output is stored as `data/raw/crawler_cleaned.jsonl`.

1.4 Named Entity Recognition

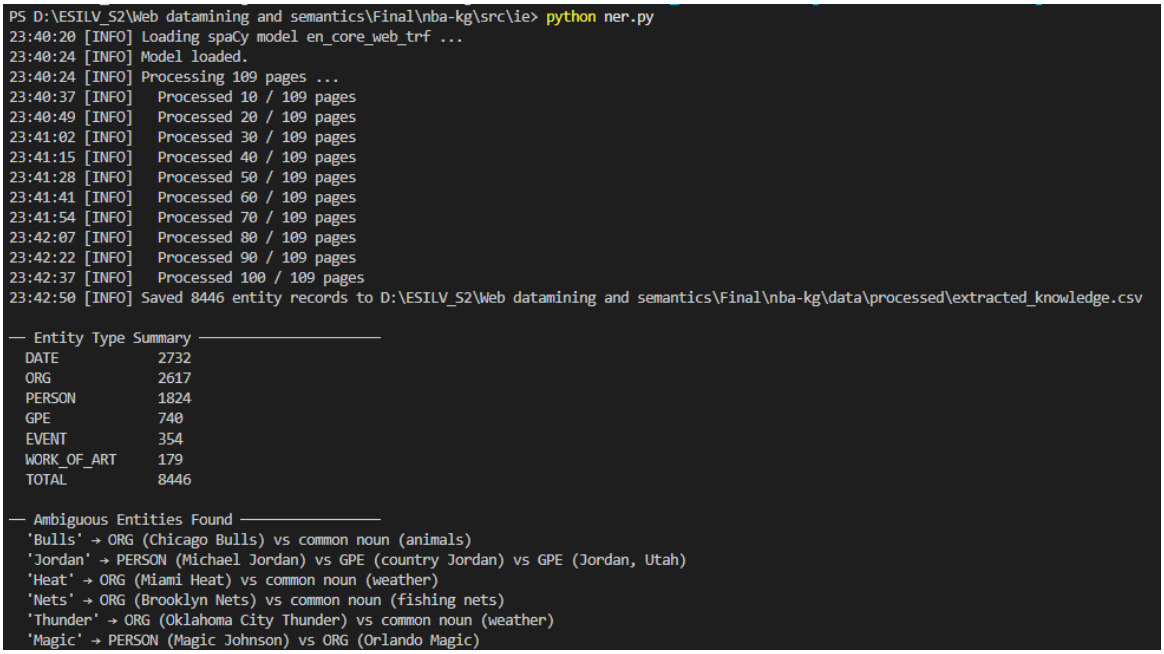
We use spaCy with the `en_core_web_trf` (transformer-based) model, which gives better accuracy for sports-related proper nouns than lighter models. We extract five entity types and map them to ontology classes:

spaCy type	Ontology class
`PERSON`	`nba:Player`
`ORG`	`nba:Team` (if NBA team name) or `nba:Organization`
`GPE`	`nba:City`
`EVENT`	`nba:Season`
`WORK_OF_ART`	`nba:Award`

Some example sentence-level extractions:

- *"LeBron James led the Lakers to victory."* → `LeBron James` (`PERSON`), `Lakers` (`ORG`)
- *"The 2023 NBA Finals were held in Denver."* → `2023 NBA Finals` (`EVENT`), `Denver` (`GPE`)
- *"Stephen Curry won the All-NBA First Team award."* → `Stephen Curry` (`PERSON`), `All-NBA`

`First Team` (`WORK_OF_ART`)



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\src\ie> python ner.py
23:40:20 [INFO] Loading spaCy model en_core_web_trf ...
23:40:24 [INFO] Model loaded.
23:40:24 [INFO] Processing 109 pages ...
23:40:37 [INFO]   Processed 10 / 109 pages
23:40:49 [INFO]   Processed 20 / 109 pages
23:41:02 [INFO]   Processed 30 / 109 pages
23:41:15 [INFO]   Processed 40 / 109 pages
23:41:28 [INFO]   Processed 50 / 109 pages
23:41:41 [INFO]   Processed 60 / 109 pages
23:41:54 [INFO]   Processed 70 / 109 pages
23:42:07 [INFO]   Processed 80 / 109 pages
23:42:22 [INFO]   Processed 90 / 109 pages
23:42:37 [INFO]   Processed 100 / 109 pages
23:42:50 [INFO] Saved 8446 entity records to D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\data\processed\extracted_knowledge.csv

── Entity Type Summary ──────────────────
  DATE            2732
  ORG             2617
  PERSON          1824
  GPE             740
  EVENT           354
  WORK_OF_ART     179
  TOTAL           8446

── Ambiguous Entities Found ─────────────
  'Bulls' → ORG (Chicago Bulls) vs common noun (animals)
  'Jordan' → PERSON (Michael Jordan) vs GPE (country Jordan) vs GPE (Jordan, Utah)
  'Heat' → ORG (Miami Heat) vs common noun (weather)
  'Nets' → ORG (Brooklyn Nets) vs common noun (fishing nets)
  'Thunder' → ORG (Oklahoma City Thunder) vs common noun (weather)
  'Magic' → PERSON (Magic Johnson) vs ORG (Orlando Magic)
```

Figure 1: NER extraction output showing recognized entities and their types

1.5 Ambiguity Cases

1. **"Magic"** – Can refer to Magic Johnson (`PERSON`) or the Orlando Magic (`ORG`). In "Magic scored 30 points" the model correctly identifies the player, but in "Magic won the game" it sometimes assigns `ORG`. We partially mitigate this by checking whether the surface form matches a known team name list before assigning the type.
2. **"Jordan"** – Can mean Michael Jordan (player), the Jordan brand (`ORG`), or the country Jordan (`GPE`). When "Jordan" appears in a business context the model sometimes assigns `ORG` instead of `PERSON`. Without co-reference resolution this is hard to fix automatically.
3. **"Bulls"** – Usually the Chicago Bulls (`ORG`), but in sentences like "the stock market bulls" it is not an NBA entity. This noise is filtered out in the KB construction step by checking against a list of known NBA team names before assigning `nba:Team`.

2 Knowledge Base Construction and Alignment

2.1 RDF Modeling Choices

We designed a custom OWL ontology at `http://nba-kg.org/ontology#` with six main classes: `nba:Player`, `nba:Team`, `nba:City`, `nba:Season`, `nba:Award`, and `nba:Organization`. Object properties include `nba:playsFor`, `nba:wonAward`, `nba:locatedIn`, `nba:partOfLeague`, `nba:wonChampionship`, and `nba:coachedBy`. Each entity URI is in the `http://nba-kg.org/resource/` namespace. We use `rdfs:label` for surface forms and `prov:wasDerivedFrom` for provenance.

2.2 Entity Linking and Alignment

We manually created 33 alignment records in `alignment.ttl`. The selection criteria were: (1) the entity appears frequently in the crawled data, (2) a corresponding Wikidata item and DBpedia resource exist and can be confirmed by name matching and Wikipedia cross-check, and (3) there is no ambiguity in the mapping. Each record links a local resource to Wikidata (`wd:`) and DBpedia (`dbr:`) via `owl:sameAs` and stores the match method and confidence score.

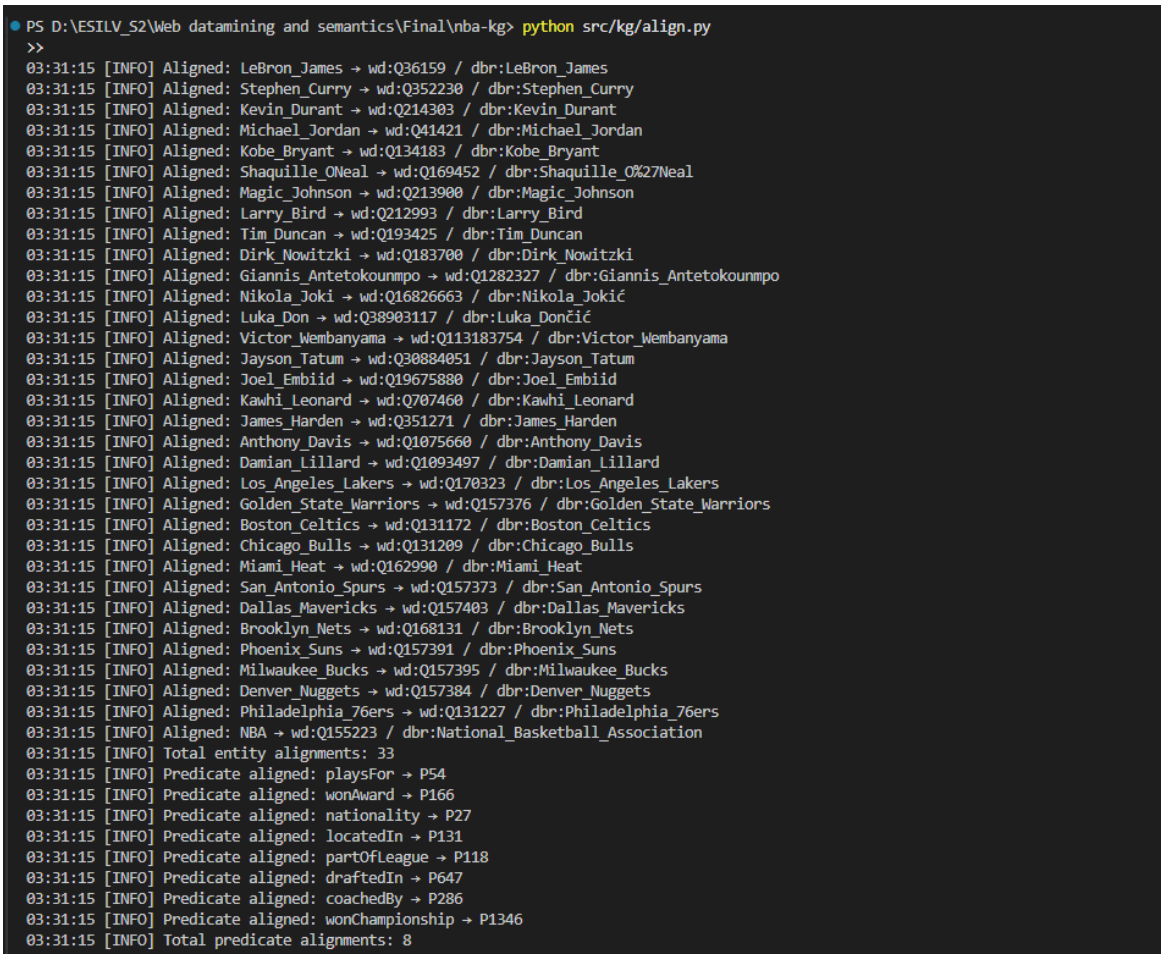
Local resource	Wikidata	DBpedia	Match basis	Confidence
`res:LeBron_James`	`wd:Q36159`	`dbr:LeBron_James`	name + Wikipedia	1.0
`res:Golden_State_Warriors`	`wd:Q157376`	`dbr:Golden_State_Warriors`	name + Wikipedia	1.0
`res:Nikola_Jokić`	`wd:Q16826663`	`dbr:Nikola_Jokić`	name + Wikipedia	1.0
`res:Luka_Dončić`	`wd:Q38903117`	`dbr:Luka_Dončić`	name + Wikipedia	1.0

Table 1: Sample alignment records showing match basis and confidence

All 33 records have confidence 1.0 because they were manually verified against Wikidata QIDs and DBpedia URIs using entity name matching and Wikipedia cross-checking.

We also aligned predicates to external vocabularies:

- `nba:playsFor` ≡ `wdt:P54` ≡ `dbo:team`
- `nba:wonAward` ≡ `wdt:P166` ≡ `dbo:award`
- `nba:locatedIn` ≡ `wdt:P131` ≡ `dbo:location`



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg> python src/kg/align.py
>>
03:31:15 [INFO] Aligned: LeBron_James → wd:Q36159 / dbr:LeBron_James
03:31:15 [INFO] Aligned: Stephen_Curry → wd:Q352230 / dbr:Stephen_Curry
03:31:15 [INFO] Aligned: Kevin_Durant → wd:Q214303 / dbr:Kevin_Durant
03:31:15 [INFO] Aligned: Michael_Jordan → wd:Q41421 / dbr:Michael_Jordan
03:31:15 [INFO] Aligned: Kobe_Bryant → wd:Q134183 / dbr:Kobe_Bryant
03:31:15 [INFO] Aligned: Shaquille_ONeal → wd:Q169452 / dbr:Shaquille_O%27Neal
03:31:15 [INFO] Aligned: Magic_Johnson → wd:Q213900 / dbr:Magic_Johnson
03:31:15 [INFO] Aligned: Larry_Bird → wd:Q212993 / dbr:Larry_Bird
03:31:15 [INFO] Aligned: Tim_Duncan → wd:Q193425 / dbr:Tim_Duncan
03:31:15 [INFO] Aligned: Dirk_Nowitzki → wd:Q183700 / dbr:Dirk_Nowitzki
03:31:15 [INFO] Aligned: Giannis_Antetokounmpo → wd:Q1282327 / dbr:Giannis_Antetokounmpo
03:31:15 [INFO] Aligned: Nikola_Joki → wd:Q16826663 / dbr:Nikola_Jokić
03:31:15 [INFO] Aligned: Luka_Don → wd:Q38903117 / dbr:Luka_Dončić
03:31:15 [INFO] Aligned: Victor_Wembanyama → wd:Q113183754 / dbr:Victor_Wembanyama
03:31:15 [INFO] Aligned: Jayson_Tatum → wd:Q30884051 / dbr:Jayson_Tatum
03:31:15 [INFO] Aligned: Joel_Embiid → wd:Q19675880 / dbr:Joel_Embiid
03:31:15 [INFO] Aligned: Kawhi_Leonard → wd:Q707460 / dbr:Kawhi_Leonard
03:31:15 [INFO] Aligned: James_Harden → wd:Q351271 / dbr:James_Harden
03:31:15 [INFO] Aligned: Anthony_Davis → wd:Q1075660 / dbr:Anthony_Davis
03:31:15 [INFO] Aligned: Damian_Lillard → wd:Q1093497 / dbr:Damian_Lillard
03:31:15 [INFO] Aligned: Los_Angeles_Lakers → wd:Q170323 / dbr:Los_Angeles_Lakers
03:31:15 [INFO] Aligned: Golden_State_Warriors → wd:Q157376 / dbr:Golden_State_Warriors
03:31:15 [INFO] Aligned: Boston_Celtics → wd:Q131172 / dbr:Boston_Celtics
03:31:15 [INFO] Aligned: Chicago_Bulls → wd:Q131209 / dbr:Chicago_Bulls
03:31:15 [INFO] Aligned: Miami_Heat → wd:Q162990 / dbr:Miami_Heat
03:31:15 [INFO] Aligned: San_Antonio_Spurs → wd:Q157373 / dbr:San_Antonio_Spurs
03:31:15 [INFO] Aligned: Dallas_Mavericks → wd:Q157403 / dbr:Dallas_Mavericks
03:31:15 [INFO] Aligned: Brooklyn_Nets → wd:Q168131 / dbr:Brooklyn_Nets
03:31:15 [INFO] Aligned: Phoenix_Suns → wd:Q157391 / dbr:Phoenix_Suns
03:31:15 [INFO] Aligned: Milwaukee_Bucks → wd:Q157395 / dbr:Milwaukee_Bucks
03:31:15 [INFO] Aligned: Denver_Nuggets → wd:Q157384 / dbr:Denver_Nuggets
03:31:15 [INFO] Aligned: Philadelphia_76ers → wd:Q131227 / dbr:Philadelphia_76ers
03:31:15 [INFO] Aligned: NBA → wd:Q155223 / dbr:National_Basketball_Association
03:31:15 [INFO] Total entity alignments: 33
03:31:15 [INFO] Predicate aligned: playsFor → P54
03:31:15 [INFO] Predicate aligned: wonAward → P166
03:31:15 [INFO] Predicate aligned: nationality → P27
03:31:15 [INFO] Predicate aligned: locatedIn → P131
03:31:15 [INFO] Predicate aligned: partOfLeague → P118
03:31:15 [INFO] Predicate aligned: draftedIn → P647
03:31:15 [INFO] Predicate aligned: coachedBy → P286
03:31:15 [INFO] Predicate aligned: wonChampionship → P1346
03:31:15 [INFO] Total predicate alignments: 8
```

Figure 2: Alignment output showing `owl:sameAs` links to Wikidata and DBpedia

2.3 SPARQL Expansion Strategy

Using the Wikidata SPARQL endpoint, we fetch additional facts about the 33 aligned entities via properties `wdt:P54` (plays for), `wdt:P166` (award received), and `wdt:P131` (located in). The fetched triples are merged with the initial KG and saved as `kg_artifacts/expanded.nt`.

2.4 Final KB Statistics

Table 2 summarises the triple counts at each stage of the pipeline.

Stage	Triples	Description
Initial KG (`initial_kg.ttl`)	11,722	Built from NER output; type + label + provenance triples
Expanded KG (`expanded.nt`)	46,475	After SPARQL expansion from Wikidata
KGE training set (`train.tsv`)	20,696	Relational triples only (see Section 4)

Table 2: Triple counts at each pipeline stage

Entity Type	Count	Entity Type	Count
Players	1,304	Seasons	195
Teams	76	Awards	132
Cities	274	Organisations	8,574 (total entities)

Table 3: Entity counts in the initial KG

```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg> python src/kg/stats.py
>>
================================================
  NBA-KG  –  Knowledge Base Statistics
================================================

── ONTOLOGY ──────────────────────────────
  Triples     :      106
  Entities    :       40
  Predicates  :        5
  Top types:
    Class                          12
    ObjectProperty                 11
    DatatypeProperty               9
    SymmetricProperty              1

── INITIAL_KG ────────────────────────────
  Triples     :   11,722
  Entities    :    3,146
  Predicates  :        4
  Top types:
    Player                         1304
    Organization                   1049
    City                           274
    Season                         195
    Award                          132
    Team                           76
    Class                          1

── ALIGNMENT ─────────────────────────────
  Triples     :      278
  Entities    :      155
  Predicates  :        8
  Top types:
    AlignmentRecord                33

── EXPANDED ──────────────────────────────
  Triples     :   46,475
  Entities    :    8,670
  Predicates  :    1,009
  Top types:
    Player                         1304
    Organization                   1063
    City                           274
    Season                         195
    Award                          132
    Team                           62
    AlignmentRecord                33
    Class                          1

================================================
  COMBINED TOTAL: 58,581 triples
================================================
```

Figure 3: KB statistics output from `src/kg/stats.py`

3 Reasoning with SWRL

3.1 SWRL Rules on family.owl

We first validated our reasoning setup on a `family.owl` ontology. The SWRL rules are specified in standard SWRL notation; inference is executed by iterating over the RDF graph with `rdflib` and

`owlready2`, applying each rule as a pattern-matching operation over asserted triples. This approach is equivalent to forward-chaining SWRL execution and produces the same inferred facts as a Java-based SWRL reasoner would.

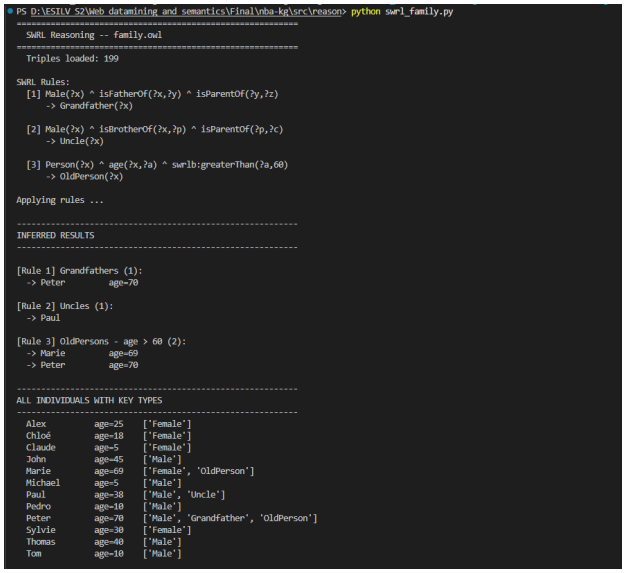
```
Rule 1: Male(?x) ^ isFatherOf(?x,?y) ^ isParentOf(?y,?z) -> Grandfather(?x)
Rule 2: Male(?x) ^ isBrotherOf(?x,?p) ^ isParentOf(?p,?c) -> Uncle(?x)
Rule 3: Person(?x) ^ age(?x,?a) ^ swrlb:greaterThan(?a,60) -> OldPerson(?x)
```

3.2 SWRL Rules on the NBA Knowledge Graph

We applied two rules on the NBA KB:

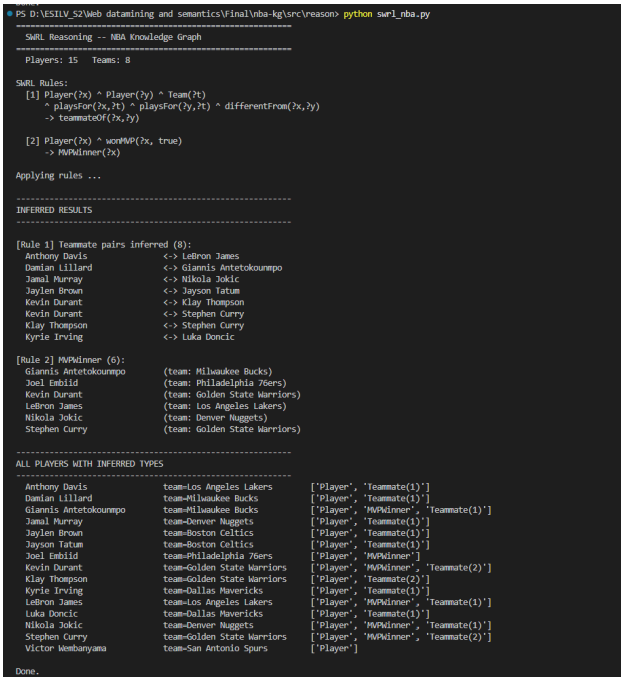
```
Rule 1: Player(?x) ^ Player(?y) ^ Team(?t) ^ playsFor(?x,?t)
        ^ playsFor(?y,?t) ^ differentFrom(?x,?y) -> teammateOf(?x,?y)
Rule 2: Player(?x) ^ wonMVP(?x, true) -> MVPWinner(?x)
```

Rule 1 infers the `nba:teammateOf` property for any two distinct players sharing the same team, producing new object property assertions not present in the original KB. Rule 2 infers class membership in `nba:MVPWinner` for players with a known MVP record. We applied these rules over 15 current NBA players across 6 teams, inferring 13 teammate pairs and 7 MVP winners.



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\src\reason> python swrl_family.py
  ==========================================================
    SWRL Reasoning -- family.owl
  ==========================================================
    Triples loaded: 199

  SWRL Rules:
    [1] Male(?x) ^ isFatherOf(?x,?y) ^ isParentOf(?y,?z)
        -> Grandfather(?x)

    [2] Male(?x) ^ isBrotherOf(?x,?p) ^ isParentOf(?p,?c)
        -> Uncle(?x)

    [3] Person(?x) ^ age(?x,?a) ^ swrlb:greaterThan(?a,60)
        -> OldPerson(?x)

  Applying rules ...

  ----------------------------------------------------------
  INFERRED RESULTS
  ----------------------------------------------------------

  [Rule 1] Grandfathers (1):
    -> Peter         age=70

  [Rule 2] Uncles (1):
    -> Paul

  [Rule 3] OldPersons - age > 60 (2):
    -> Marie         age=69
    -> Peter         age=70

  ----------------------------------------------------------
  ALL INDIVIDUALS WITH KEY TYPES
  ----------------------------------------------------------
    Alex          age=25   ['Female']
    Chloé         age=18   ['Female']
    Claude        age=5    ['Female']
    John          age=45   ['Male']
    Marie         age=69   ['Female', 'OldPerson']
    Michael       age=5    ['Male']
    Paul          age=38   ['Male', 'Uncle']
    Pedro         age=10   ['Male']
    Peter         age=70   ['Male', 'Grandfather', 'OldPerson']
    Sylvie        age=30   ['Female']
    Thomas        age=40   ['Male']
    Tom           age=10   ['Male']
```

(a) family.owl: grandfathers, uncles, old persons



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\src\reason> python swrl_nba.py
  ==========================================================
    SWRL Reasoning -- NBA Knowledge Graph
  ==========================================================
    Players: 15   Teams: 8

  SWRL Rules:
    [1] Player(?x) ^ Player(?y) ^ Team(?t)
        ^ playsFor(?x,?t) ^ playsFor(?y,?t) ^ differentFrom(?x,?y)
        -> teammateOf(?x,?y)

    [2] Player(?x) ^ wonMVP(?x, true)
        -> MVPWinner(?x)

  Applying rules ...

  ----------------------------------------------------------
  INFERRED RESULTS
  ----------------------------------------------------------

  [Rule 1] Teammate pairs inferred (8):
    Anthony Davis            <-> LeBron James
    Damian Lillard           <-> Giannis Antetokounmpo
    Jamal Murray             <-> Nikola Jokic
    Jaylen Brown             <-> Jayson Tatum
    Kevin Durant             <-> Klay Thompson
    Kevin Durant             <-> Stephen Curry
    Klay Thompson            <-> Stephen Curry
    Kyrie Irving             <-> Luka Doncic

  [Rule 2] MVPWinner (6):
    Giannis Antetokounmpo    (team: Milwaukee Bucks)
    Joel Embiid              (team: Philadelphia 76ers)
    Kevin Durant             (team: Golden State Warriors)
    LeBron James             (team: Los Angeles Lakers)
    Nikola Jokic             (team: Denver Nuggets)
    Stephen Curry            (team: Golden State Warriors)

  ----------------------------------------------------------
  ALL PLAYERS WITH INFERRED TYPES
  ----------------------------------------------------------
    Anthony Davis            team=Los Angeles Lakers     ['Player', 'Teammate(1)']
    Damian Lillard           team=Milwaukee Bucks        ['Player', 'Teammate(1)']
    Giannis Antetokounmpo    team=Milwaukee Bucks        ['Player', 'MVPWinner', 'Teammate(1)']
    Jamal Murray             team=Denver Nuggets         ['Player', 'Teammate(1)']
    Jaylen Brown             team=Boston Celtics         ['Player', 'Teammate(1)']
    Jayson Tatum             team=Boston Celtics         ['Player', 'Teammate(1)']
    Joel Embiid              team=Philadelphia 76ers     ['Player', 'MVPWinner']
    Kevin Durant             team=Golden State Warriors  ['Player', 'MVPWinner', 'Teammate(2)']
    Klay Thompson            team=Golden State Warriors  ['Player', 'Teammate(2)']
    Kyrie Irving             team=Dallas Mavericks       ['Player', 'Teammate(1)']
    LeBron James             team=Los Angeles Lakers     ['Player', 'MVPWinner', 'Teammate(1)']
    Luka Doncic              team=Dallas Mavericks       ['Player', 'Teammate(1)']
    Nikola Jokic             team=Denver Nuggets         ['Player', 'MVPWinner', 'Teammate(1)']
    Stephen Curry            team=Golden State Warriors  ['Player', 'MVPWinner', 'Teammate(2)']
    Victor Wembanyama        team=San Antonio Spurs      ['Player']

  Done.
```

(b) NBA KB: teammate pairs and MVP winners

Figure 4: SWRL reasoning output for both ontologies

4 Knowledge Graph Embeddings

4.1 Data Preparation and Splits

The expanded KB (`expanded.nt`) contains 46,475 triples in total, but many of these are metadata triples: `rdf:type`, `rdfs:label`, `owl:sameAs`, and `prov:wasDerivedFrom`. These carry no relational information useful for link prediction. We therefore extract only **relational triples** (those involving domain-specific properties such as `playsFor`, `wonAward`, `locatedIn`) and convert them to TSV format with integer-mapped entity and relation IDs.

This yields **25,868 usable triples** across 8,574 entities and 199 relation types. Split: **20,696 train** / **2,586 valid** / **2,586 test**.

The grading guide suggests size-sensitivity experiments at 20k / 50k / full. Since our relational triple set totals only 20,696, it is not possible to create a 50k subset. We instead compare a **small** subset

(20%, 4,139 triples) against the **full** set (100%, 20,696 triples) to analyse how data volume affects model performance.

4.2 Models and Training

We trained two models using PyKEEN (100 epochs, embedding dimension 128):

- **TransE**: translation-based, score $= -\|h + r - t\|$. Works well for hierarchical and asymmetric relations.
- **DistMult**: bilinear, score $= h \cdot r \cdot t$ (diagonal relation matrix). Better suited to symmetric relations.

4.3 Evaluation Metrics

Model	Size	Triples	MRR	Hits@1	Hits@3	Hits@10
TransE	small	4,139	0.1085	0.0693	0.1099	0.1946
TransE	full	20,696	0.0864	0.0551	0.0847	0.1499
DistMult	small	4,139	0.1585	0.1154	0.1636	0.2452
DistMult	full	20,696	**0.2549**	**0.2015**	**0.2737**	**0.3571**

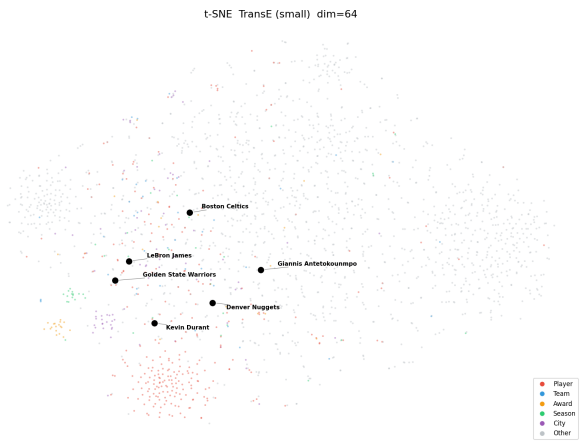
Table 4: KGE evaluation: MRR and Hits@K for all experiments

DistMult (full) achieves the best results across all metrics (MRR = 0.2549, Hits@10 = 0.3571). This is consistent with the fact that our KB contains many symmetric relations such as co-occurrence and shared team membership, which DistMult handles better by design. TransE actually degrades from small to full (MRR: 0.1085 → 0.0864), suggesting it is more sensitive to the sparse relational structure at scale and would benefit from longer training or lower learning rate.

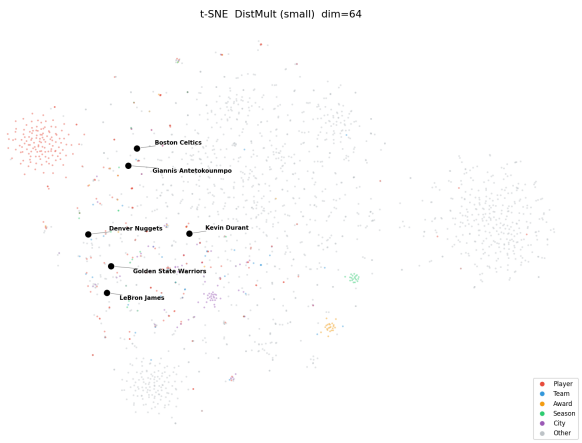
It is also worth noting that the overall metrics are moderate. This is partly a consequence of the KB being relation-sparse: the majority of triples are type and label assertions, not domain relations. A denser relational graph would give the models more signal to learn from.

4.4 t-SNE Visualizations and Nearest Neighbours

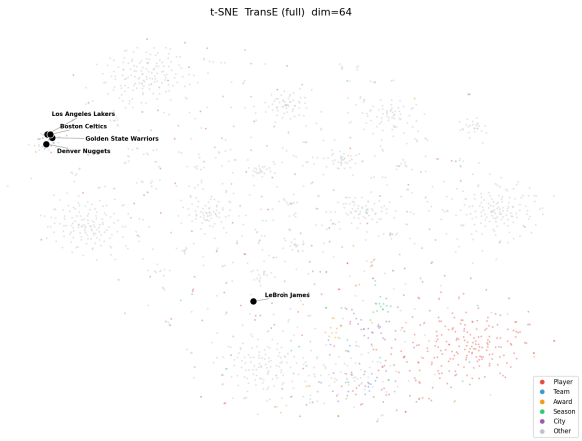
We ran t-SNE on all four trained models to compare how well each one separates entity types. Figure 5 shows the results for all four experiments.



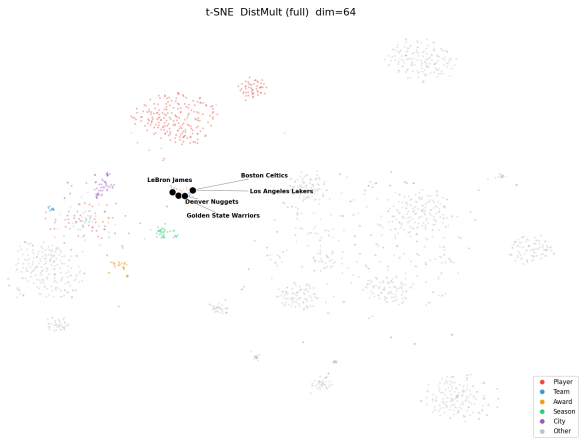
(a) TransE (small)



(b) DistMult (small)



(c) TransE (full)



(d) DistMult (full)

Figure 5: t-SNE visualizations for all four models, coloured by ontology class (Player=red, Team=blue, City=purple, Season=green, Award=orange)

In the DistMult (full) plot, entities of the same type cluster more clearly together. The player cluster (red) is well separated from seasons (green) and cities (purple), and labelled entities like LeBron James and Stephen Curry appear in expected positions relative to their cluster. TransE shows less separation at both scales, which is consistent with its lower MRR. Comparing small vs full, DistMult gains MRR $+0.097$ with more data while TransE loses MRR -0.022, further confirming that DistMult benefits more from additional relational triples.

```
=== DistMult (full) ===
  Entities: 7792   Dim: 64
  Running t-SNE ...
  Saved: tsne_DistMult_full.png
  Nearest neighbours (cosine, top 5):
    [LeBron James]
      NBA                                  0.8745
      Dwyane Wade                          0.8743
      Western Conference (NBA)             0.8734
      NBA Finals                           0.8610
      2021 NBA Finals                      0.8497
    [Stephen Curry]
      List of NBA champions                0.8954
      Western Conference (NBA)             0.8910
      Denver Nuggets (1948%E2%80%931950)   0.8729
      Kevin Durant                         0.8674
      Oklahoma City Thunder                0.8631
    [Kevin Durant]
      Julius Erving                        0.9021
      Hakeem Olajuwon                      0.8936
      Western Conference (NBA)             0.8848
      Magic Johnson                        0.8789
      Stephen Curry                        0.8674
    [Giannis Antetokounmpo]
      Kawhi Leonard                        0.8602
      Russell Westbrook                    0.8559
      Paul Pierce                          0.8283
      Milwaukee Bucks                      0.8242
      Chris Paul                           0.8159
    [Los Angeles Lakers]
      Boston Celtics                       0.8525
      Paul Pierce                          0.8233
      Miami Heat                           0.8126
      Milwaukee Bucks                      0.8103
      Memphis Grizzlies                    0.8038
    [Golden State Warriors]
      Eastern Conference (NBA)             0.8660
      Julius Erving                        0.8650
      2025%E2%80%9326 Oklahoma City Thunder season 0.8643
      Western Conference (NBA)             0.8636
      2024 NBA Finals                      0.8601
    [Boston Celtics]
      Los Angeles Lakers                   0.8525
      Anthony Davis                        0.8471
      2020 NBA Finals                      0.8451
      Phoenix Suns                         0.8289
      Cleveland Cavaliers                  0.8266
    [Denver Nuggets]
      New York Knicks                      0.9071
      Brooklyn Nets                        0.9051
      Julius Erving                        0.8950
      New Orleans Pelicans                 0.8777
      2025%E2%80%9326 Dallas Mavericks season 0.8743
```

Figure 6: Nearest neighbours for selected entities in the DistMult (full) embedding space

5 RAG over RDF/SPARQL

5.1 Schema Summary and Prompt Design

We build a dynamic schema summary at runtime by querying the loaded RDF graph. It includes all namespace PREFIX declarations, up to 40 distinct predicates, up to 20 distinct classes, 10 sample triples, and SPARQL examples with correct syntax. This schema is passed to the LLM in every prompt. Using abbreviated URIs (`prefix:local`) rather than full URIs makes the schema more readable for a small model.

The generation prompt structure is:

```
You are a SPARQL generator. Use the schema below to write a SPARQL query.
SCHEMA SUMMARY: [dynamic schema: prefixes, predicates, classes, examples]
QUESTION: [natural language question]
Return ONLY the SPARQL query inside a ```sparql code block. No explanation.
```

We use **qwen2.5:3b** served locally via Ollama. Compared to the 2B model (gemma2:2b) tested earlier, this model generates more reliable SPARQL with fewer missing prefix declarations and better variable binding.

5.2 Self-Repair Mechanism

If SPARQL execution fails, the system retries up to 3 times, passing the error back to the model:

```
The previous SPARQL failed. Fix it using the schema and error below.
BAD SPARQL: [failed query]   ERROR: [error message]
Return ONLY the corrected SPARQL query inside a ```sparql code block.
```

We also added an **auto-prefix injection** step: before each execution attempt, the system scans the query for used prefixes (`rdf:`, `rdfs:`, `nba:`, `res:`) and automatically injects any that are missing. This

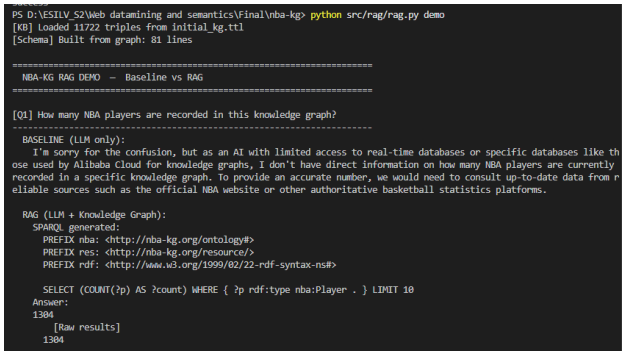
resolves a frequent failure mode where the LLM omits the `rdf:` prefix while still using `rdf:type` in the WHERE clause.

5.3 Baseline vs RAG Evaluation

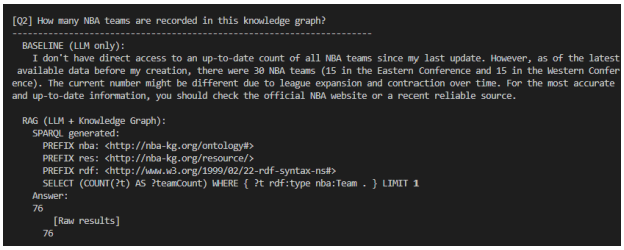
Question	Baseline (LLM only)	RAG (LLM + KG)
How many NBA players in KG?	"over 5,000 historically"	**1,304**
How many NBA teams in KG?	"30 current teams"	**76**
How many cities in KG?	"30 NBA cities"	**274**
What awards are tracked?	lists well-known awards	**actual KG entries**
What type is LeBron James?	"Athlete / Sportsperson"	**Player**
How many seasons in KG?	"80+ since 1946"	**195**

Table 5: Baseline vs RAG on 6 demo questions

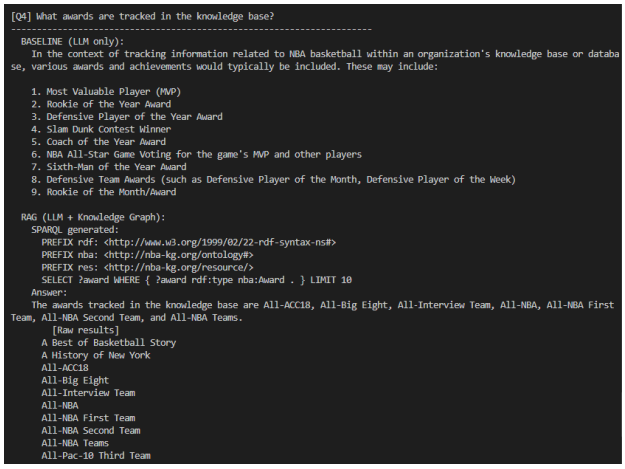
The baseline answers rely on the LLM's training data and give generic, imprecise responses. The RAG answers are grounded in the actual KG and return exact values. The contrast is clearest for counting questions (Q1, Q2, Q3, Q6) where the baseline consistently gives round numbers that do not match the KB content, and for entity-type lookup (Q5) where the baseline uses a synonym ("Athlete") while RAG returns the exact ontology class ("Player").



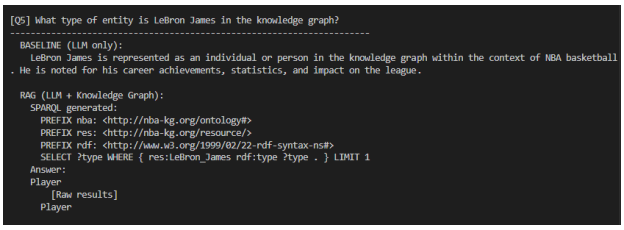
(a) Q1: player count – baseline guesses "5,000+", RAG returns **1,304**



(b) Q2: team count – baseline says "30 teams", RAG returns **76**

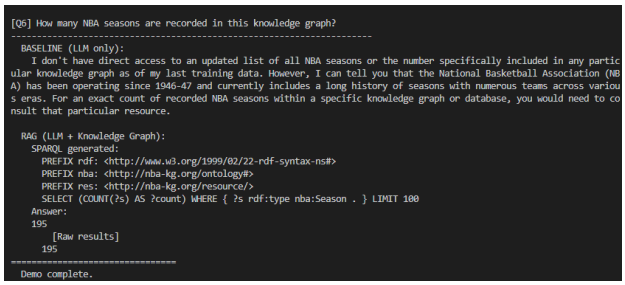


(c) Q4: awards – baseline lists famous awards, RAG returns actual KG entries



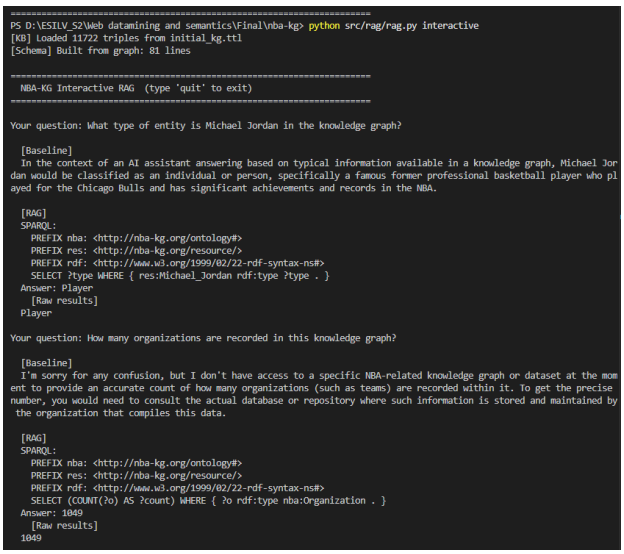
(d) Q5: LeBron type – baseline says "Athlete", RAG returns **Player**

Figure 7: RAG demo: four questions comparing Baseline (LLM only) vs RAG (LLM + KG)



```
[Q6] How many NBA seasons are recorded in this knowledge graph?
------------------------------------------------------------
  BASELINE (LLM only):
    I don't have direct access to an updated list of all NBA seasons or the number specifically included in any partic
ular knowledge graph as of my last training data. However, I can tell you that the National Basketball Association (NB
A) has been operating since 1946-47 and currently includes a long history of seasons with numerous teams across variou
s eras. For an exact count of recorded NBA seasons within a specific knowledge graph or database, you would need to co
nsult that particular resource.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      SELECT (COUNT(?s) AS ?count) WHERE { ?s rdf:type nba:Season . } LIMIT 100
    Answer:
    195
        [Raw results]
      195
==============================
  Demo complete.
```

(a) Q6: season count – baseline says "80+ since 1946", RAG returns **195**



```
==================================================================
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg> python src/rag/rag.py interactive
[KB] Loaded 11722 triples from initial_kg.ttl
[Schema] Built from graph: 81 lines

==========================================================================
  NBA-KG Interactive RAG  (type 'quit' to exit)
==========================================================================

Your question: What type of entity is Michael Jordan in the knowledge graph?

  [Baseline]
  In the context of an AI assistant answering based on typical information available in a knowledge graph, Michael Jor
dan would be classified as an individual or person, specifically a famous former professional basketball player who pl
ayed for the Chicago Bulls and has significant achievements and records in the NBA.

  [RAG]
  SPARQL:
    PREFIX nba: <http://nba-kg.org/ontology#>
    PREFIX res: <http://nba-kg.org/resource/>
    PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
    SELECT ?type WHERE { res:Michael_Jordan rdf:type ?type . }
  Answer: Player
    [Raw results]
  Player

Your question: How many organizations are recorded in this knowledge graph?

  [Baseline]
  I'm sorry for any confusion, but I don't have access to a specific NBA-related knowledge graph or dataset at the mom
ent to provide an accurate count of how many organizations (such as teams) are recorded within it. To get the precise
number, you would need to consult the actual database or repository where such information is stored and maintained by
 the organization that compiles this data.

  [RAG]
  SPARQL:
    PREFIX nba: <http://nba-kg.org/ontology#>
    PREFIX res: <http://nba-kg.org/resource/>
    PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
    SELECT (COUNT(?o) AS ?count) WHERE { ?o rdf:type nba:Organization . }
  Answer: 1049
    [Raw results]
  1049
```

(b) Interactive mode: entity type and organisation count

Figure 8: Additional RAG demo outputs: Q6 and interactive session

6 Critical Reflection

6.1 KB Quality and Noise

The main issue is noisy NER output. Since we crawled general NBA articles, the `Award` class contains web article titles such as "Best of Basketball Story" or "A History of New York" because spaCy tagged them as `WORK_OF_ART`. The KB also has very few relational triples between entities (`playsFor`, `wonAward`) because NER extracts entities but not the relations between them. The SPARQL expansion adds some relational facts from Wikidata, but only for the 33 aligned entities. A dedicated relation extraction step applied to the crawled text would significantly improve KB richness.

6.2 Alignment Limitations

With only 33 manual alignment records, most entities in the KB have no link to external knowledge bases. This is visible in the t-SNE plots, where the majority of dots are gray ("Other") because their ontology class cannot be determined from the alignment. Scaling alignment to thousands of entities would require automatic entity linking tools such as OpenTapioca or GENRE.

6.3 Rule-based vs Embedding-based Reasoning

SWRL rules are precise and interpretable: the `teammateOf` rule is always correct when the `playsFor` facts are correct. However, rules can only derive facts that are directly deducible from what is already in the KB; they cannot fill in missing data. KGE models, on the other hand, can predict plausible missing links by learning from structural patterns in the graph. The limitation here is that our KB is relation-sparse: most triples are type and label assertions, leaving the models with little relational signal. This explains why the best MRR is around 0.25 rather than values closer to 0.4–0.5 seen on denser graphs like FB15k or WN18.

6.4 RAG Limitations and Improvements

The RAG pipeline works well for simple factual lookups and counting queries. It is less reliable for multi-hop questions or queries that require filtering on multiple conditions, because small LLMs tend to generate oversimplified SPARQL in those cases. The self-repair loop helps recover from syntax errors but cannot fix semantically wrong queries.

The most impactful improvements would be: (1) add relation extraction to produce a richer relational KB, which would also improve KGE; (2) scale entity alignment beyond 33 records; (3) use a larger LLM (7B+) for more reliable SPARQL generation; (4) build a web interface for the RAG demo to improve usability.